Applied Soft Computing Journal 93 (2020) 106401

Contents lists available at ScienceDirect

Applied Soft Computing Journal

journal homepage: www.elsevier.com/locate/asoc

Using Deep Learning for price prediction by exploiting stationary limit order book features

Avraam Tsantekidis [a,*], Nikolaos Passalis [b], Anastasios Tefas [a], Juho Kanniainen [b], Moncef Gabbouj [b], Alexandros Iosifidis [c]

[a] Department of Informatics, Aristotle University of Thessaloniki, Thessaloniki, Greece
[b] Computing sciences, Tampere University, Tampere, Finland
[c] Department of Engineering, Electrical and Computer Engineering, Aarhus University, Denmark

ARTICLE INFO

Article history:
Received 16 October 2018
Received in revised form 12 March 2020
Accepted 11 May 2020
Available online 18 May 2020

Keywords:
Limit order book
Stationary features
Price forecasting
Deep Learning

ABSTRACT

The recent surge in Deep Learning (DL) research of the past decade has successfully provided solution to many difficult problems. The field of Quantitative analysis has been slowly adapting the new methods to its problems, but due to problems such as the non-stationary nature of financial data, significant challenges must be overcome before DL is fully utilized. In this work a new method to construct stationary features is proposed such that allows DL models to be applied effectively. These features are thoroughly tested on the task of predicting mid price movements of the Limit Order Book. Several DL models are evaluated such as recurrent Long Short Term Memory (LSTM) networks and Convolutional Neural Networks (CNN). Finally a novel model that combines the ability of the CNN to extract useful features and the ability of LSTMs' to analyse time series, is proposed and evaluated. The combined model is able to outperform the individual LSTM and CNN models in the prediction horizons that are tested.

© 2020 Elsevier B.V. All rights reserved.

1. Introduction

In the last decade Machine Learning (ML) has been rapidly evolving due to the profound performance improvements that Deep Learning (DL) has ushered. Deep Learning has outperformed previous state-of-the-art methods in many fields of Machine Learning, such as Natural Language Processing (NLP) [1], image processing [2] and speech generation [3]. As the number of new methods incorporating Deep Learning in many scientific fields increase, the proposed solutions begin to span across other disciplines where Machine Learning was used in a limited capacity. One such example is the quantitative analysis of the stock markets and the usage of Machine Learning to predict price movements or the volatility of the future prices or the detection of anomalous events in the markets.

In the field of quantitative analysis, the mathematical modelling of the markets has been the de facto approach to model stock price dynamics for trading, market making, hedging, and risk management. By utilizing a time series of values, such as the price fluctuations of financial products being traded in the markets, one can construct statistical models which can assist in the extraction of useful information about the current state of the market and a set of probabilities for possible future states, such as price or volatility changes. Many models, such as the Black–Scholes–Merton model [4], attempted to mathematically deduce the price of options and can be used to provide useful indications of future price movements.

However, at some point as more market participants started using the same model the behaviour of the price changed to the point that it could no longer be taken advantage of. Newer models, such as the stochastic modelling of limit order book dynamics [5], the jump-diffusion processes for stock dynamics [6] and volatility estimation of market microstructure noise [7] have been attempts predict multiple aspects of the financial markets. However such models are designed to be tractable, even at the cost of reliability and accuracy, and thus they do not necessarily fit empirical data very well.

These properties put these models at a disadvantage, since the financial markets very frequently exhibit irrational behaviour, mainly due to the large influence of human activity, which frequently causes such "static" models to fail. Combining Machine Learning models with handcrafted features usually improves the forecasting abilities of such models, by overcoming some of the aforementioned limitations, and improving predictions about various aspects of financial markets. This led many organizations that participate in the Financial Markets, such as Hedge Funds

* Corresponding author.
E-mail addresses: avraamt@csd.auth.gr (A. Tsantekidis), passalis@csd.auth.gr (N. Passalis), tefas@aiia.csd.auth.gr (A. Tefas), juho.kanniainen@tut.fi (J. Kanniainen), moncef.gabbouj@tut.fi (M. Gabbouj), alexandros.iosifidis@eng.au.dk (A. Iosifidis).

https://doi.org/10.1016/j.asoc.2020.106401
1568-4946/© 2020 Elsevier B.V. All rights reserved.

and investment firms, to increasingly use ML models, along with the conventional mathematical models, to make crucial decisions.

Furthermore, the introduction of electronic trading, that also led to the automation of trading operations, has magnified the volume of exchanges, producing a wealth of data. Deep Learning models are perfect candidates for analysing such amounts of data, since they perform significantly better than the conventional Machine Learning methodologies when a large amount of data is available. This is one of the reasons that Deep Learning is starting to have a major role in analysing the data coming from financial exchanges.

The most detailed type of data that financial exchanges are gathering is the comprehensive logs of every submitted order and event that is happening within their internal matching engine. This log can be used to reconstruct the Limit Order Book (LOB), which is explained further in Section 3.1. A basic task that can arise from this data is the prediction of future price movements of an asset by examining the current and past supply and demand of Limit Orders. This type of comprehensive logs kept by the exchanges is excessively large and traditional Machine Learning techniques, such as Support Vector Machines (SVMs) [8], usually cannot be applied out-of-the-box. Utilizing this kind of data directly with existing Deep Learning methods is also not possible due to their non-stationary nature. Prices fluctuate and suffer from stochastic drift, so in order for them to be effectively utilized by DL methods a preprocessing step is required to generate stationary features from them.

The main contribution of this work is proposing a set of stationary features that can be readily extracted from the Limit Order Book, allowing for significantly improving the performance of established DL models, such as CNNs and LSTMs. Furthermore, we also discuss and demonstrate how the performance of such models can be further improved by (a) employing a Deep Learning model that combines the feature extraction ability of Convolutional Neural Networks (CNNs) with the Long Short Term Memory (LSTM) networks' power to analyse time series and (b) employing a simple, yet effective, approach for initializing the memory of LSTMs using a burn-in period before training the models.

In Section 2 related work which employs ML models on financial data is briefly presented. Then, the dataset used is described in detail in Section 3. In Section 4 the proposed stationary feature extraction methodology is presented in detail, while in Section 5 the proposed Deep Learning methods are described. In Section 6 the experimental evaluation and comparisons are provided. Finally, conclusions are drawn and future work is discussed in Section 7.

2. Related work

The task of regressing the future movements of financial assets has been the subject of many recent works such as [9–11]. Proven models such as GARCH are improved and augmented with machine learning component such as Artificial Neural Networks [12]. New hybrid models are employed along with Neural Networks to improve upon previous performance [13].

One of the most volatile financial markets is FOREX, the currency markets. In [14], neural networks are used to predict the future exchange rate of major FOREX pairs such as USD/EUR. The model is tested with different prediction steps ranging from daily to yearly which reaches the conclusion that shorter term predictions tend to be more accurate. Other financial metrics, such as cash flow prediction, are very closely correlated to price prediction.

In [15], the authors propose the "Deep Portfolio Theory" which applies autoencoders in order to produce optimal portfolios. This approach outperforms several established benchmarks, such as the Biotechnology IBB Index. Likewise in [16], another type of autoencoders, known as Restricted Boltzmann Machine (RBM), is applied to encode the end-of-month prices of stocks. Then, the model is fine-tuned to predict whether the price will move more than the median change and the direction of such movement. This strategy is able to outperform a benchmark momentum strategy in terms of annualized returns.

Another approach is to include data sources outside the financial time series, e.g., [17], where phrases related to finance, such as "mortgage" and "bankruptcy" were monitored on the Google trends platform and included as an input to a recurrent neural network along with the daily S&P 500 market fund prices. The training target is the prediction of the future volatility of the market fund's price. This approach can greatly outperform many benchmark methods, such as the autoregressive GARCH and Lasso techniques.

The surge of DL methods has dramatically improved the performance over many conventional machine learning methods on tasks, such as speech recognition [18], image captioning [19,20], and question answering [21]. The most important building blocks of DL are the Convolutional Neural Networks (CNN) [22], and the Recurrent Neural Networks (RNNs). Also worth mentioning is the improvement of RNNs with the introduction of Long Short-Term Memory Units (LSTMs) [23], which has made the analysis of time series using DL easier and more performant.

Unfortunately DL methods are prone to overfit especially in tasks such as price regression and many works exist trying to prevent such overfitting [24,25]. Some might attribute overfitting to the lack of huge amounts of data that other tasks such as image and speech processing have available to them. A very rich data source for financial forecasting is the Limit Order Book. One of the few applications of ML in high frequency Limit Order Book data is [26], where several handcrafted features are created, including price deltas, bid–ask spreads and price and volume derivatives. An SVM is then trained to predict the direction of future mid price movements using all the handcrafted features. In [27] a neural network architecture incorporating the idea of bilinear projection augmented with a temporal attention mechanism is used to predict LOB mid price.

Similarly in [28,29] utilize the Limit Order Book data along with ML methods such as multilinear methods and smart feature selection to predict the future price movements. In our previous work [30–32] we introduced a large-scale high-frequency Limit Order Book dataset, that is also used in this paper, and we employed three simple DL models, the Convolutional Neural Networks (CNN), the Long-Short Term Memory Recurrent Neural Networks (LSTM RNNs) and the Neural Bag-of-Features (N-BoF) model, to tackle the problem of forecasting the mid price movements. However, these approaches directly used the non-stationary raw Order Book data, making them vulnerable to distribution shifts and harming their ability to generalize on unseen data, as we also experimentally demonstrate in this paper.

To the best of our knowledge this is the first work that proposes a structured approach for extracting stationary price features from the Limit Order Book that can be effectively combined with Deep Learning models. We also provide an extensive evaluation of the proposed methods on a large-scale dataset with more than 4 million events. Also, a powerful model, that combines the CNN feature extraction properties with the LSTM's time series modelling capabilities, is proposed in order to improve the accuracy of predicting the price movement of stocks. The proposed combined model is also compared with the previously introduced methods using the proposed stationary price features.

3. Datasets

We utilize our main financial dataset consisting of the Limit Order Book Data to present our proposed stationary price feature and the capability of our model to forecast mid price movements. We also include a second simpler dataset of FOREX prices to forecast the price movement to reinforce the claims of our proposed approach. In this Section we present both of the datasets.

3.1. Limit Order Book data

In an order-driven financial market, a market participant can place two types of buy/sell orders. By posting a *limit order*, a trader promises to buy (sell) a certain amount of an asset at a specified price or less (more). The limit order book compromises on the valid limit order that are not executed or cancelled yet.

This Limit Order Book (LOB) contains all existing buy and sell orders that have been submitted and are awaiting to be executed. A limit order is placed on the queue at a given price level, where, in the case of standard limit orders, the execution priority at a given price level is dictated by the arrival time (first in, first out). A *market order* is an order to immediately buy/sell a certain quantity of the asset at the best available price in the limit order book. If the requested price of a limit order is far from the best prices, it may take a long time for the execution of the limit order, in which case, the order can finally be cancelled by the trader. The orders are split between two sides, the bid (buy) and the ask (sell) side. Each side contains the orders sorted by their price, in descending order for the bid side and ascending order for the ask side.

Following the notation used in [5], a price grid is defined as $\{\rho^{(1)}(t), \ldots, \rho^{(n)}(t)\}$, where $\rho^{(j)}(t) > \rho^{(i)}(t)$ for all $j > i$. The price grid contains all possible prices and each consecutive price level is incremented by a single tick from the previous price level. The state of the order book is a continuous-time process $v(t) \equiv \left(v^{(1)}(t), v^{(2)}(t), \ldots, v^{(n)}(t)\right)_{t\geq 0}$, where $|v^{(i)}(t)|$ is the number of outstanding limit orders at price $\rho^{(i)}(t)$, $1 \leq i \leq n$. If $v^{(i)}(t) < 0$, then there are $-v^{(i)}(t)$ bid orders at price $\rho^{(i)}(t)$; if $v^{(i)}(t) > 0$, then there are $v^{(i)}(t)$ ask orders at price $\rho^{(i)}(t)$. That is, $v^{(i)}(t) > 0$ refers to ask orders and $v^{(i)}(t) < 0$ bid orders.

The location of the best ask price in the price grid is defined by:

$$i_a^{(1)}(t) = \inf\{i = 1, \ldots, n\ ;\ v^{(i)}(t) > 0\},$$

and, correspondingly, the location of the best bid price is defined by:

$$i_b^{(1)}(t) = \sup\{i = 1, \ldots, n\ ;\ v^{(i)}(t) < 0\}.$$

For simplicity, we denote the best ask and bid prices as $p_a^{(1)}(t) \equiv \rho^{\left(i_a^{(1)}(t)\right)}(t)$ and $p_b^{(1)}(t) \equiv \rho^{\left(i_b^{(1)}(t)\right)}(t)$, respectively. Notice that if there are no ask (bid) orders in the book, the ask (bid) price is not defined.

More generally, given that the kth best ask and bid prices exist, their locations are denoted as $i_a^{(k)}(t) \equiv i_a(t) + k - 1$ and $i_b^{(k)}(t) \equiv i_b(t) + k - 1$. The kth best ask and bid prices are correspondingly denoted by $p_a^{(k)}(t) \equiv \rho^{\left(i_a^{(k)}(t)\right)}(t)$ and $p_b^{(k)}(t) \equiv \rho^{\left(i_b^{(k)}(t)\right)}(t)$, respectively. Correspondingly, we denote the number of outstanding limit orders at the kth best ask and bid levels by $\nu_a^{(k)}(t) \equiv v^{\left(i_a^{(k)}(t)\right)}(t)$ and $\nu_b^{(k)}(t) \equiv v^{\left(i_b^{(k)}(t)\right)}(t)$, respectively.

Limit Order Book data can be used for a variety of tasks, such as the estimation of the future price trend or the regression of useful metrics, like the price volatility. Other possible tasks may include the early prediction of anomalous events, like extreme changes in price which may indicate manipulation in the markets. These examples are a few of multiple applications which can aid investors to protect their capital when unfavourable conditions exist in the markets or, in other cases, take advantage of them to profit.

Most modern methods that utilize financial time series data employ subsampling techniques, such as the well-known OHLC (Open-High-Low-Close) candles [33], in order to reduce the number of features of each time interval. Although the OHLC candles preserve useful information, such as the market trend and movement ranges within the specified intervals, it removes possibly important microstructure information. Since the LOB is constantly receiving new orders in inconsistent intervals, it is not possible to subsample time-interval features from it in a way that preserves all the information it contains. This problem can be addressed, to some extent, using recurrent neural network architectures, such as LSTMs, that are capable of natively handling inputs of varying size. This allows to directly utilize the data fully without using a time interval-based subsampling.

The LOB data used in this work is provided by Nasdaq Nordic and consists of 10 days worth of LOB events for 5 different Finnish company stocks, namely Kesko Oyj, Outokumpu Oyj, Sampo, Rautaruukki and Wärtsilä Oyj [34,35]. The exact time period of the gathered data begins from the 1st of June 2010 to the 14th of June 2010. Also, note that trading only happens during business days.

The data consists of consecutive snapshots of the LOB state after each state altering event takes place. This event might be an order insertion, execution or cancellation and after it interacts with the LOB and change its state a snapshot of the new state is taken. The LOB depth of the data that are used is 10 for each side of Order Book, which ends up being 10 active orders (consisting of price and volume) for each side adding up to a total of 40 values for each LOB snapshot. Although the dataset tested does not span across a long period of time, the high frequency sampling that is used allows the collection of 4.5 million snapshots that can be used to train and evaluate the proposed models.

In this work the task we aim to accomplish is the prediction of price movements based on current and past changes occurring in the LOB. This problem is formally defined as follows: Let $\mathbf{x}(t) \in \mathbb{R}^q$ denote the feature vector that describes the condition of the LOB at time t for a specific stock, where q is the dimensionality of the corresponding feature vector. The direction of the mid-price of that stock is defined as $l_k(t) = \{-1, 0, 1\}$ depending on whether the mid price decreased (-1), remained stationary (0) or increased (1) after k LOB events occurred. The number of orders k is also called *prediction horizon*. We aim to learn a model $f_k(\mathbf{x}(t))$, where $f_k : \mathbb{R}^n \rightarrow \{-1, 0, 1\}$, that predicts the direction $l_k(t)$ of the mid-price after k orders. In the following Section the aforementioned features and labels, as well as the procedure to calculate them are explained in depth.

3.2. FOREX price data

Open-High-Low-Close (OHLC) prices is a subsampling technique to enable the illustration of price movements across various time scales. The data provided by the financial exchanges starts from the LOB data as presented in Section 3.1 which is the highest resolution available. The LOB event data can be subsampled to only include the events where an order has been executed and assets have been traded between participants.

Trade events mainly consist of three values, the price at which the trade was made and the amount of the traded asset that was exchanged and the exact time that it took place. In order to make the varying number of ticks across time coherent to observers of the price, it is subsampled using the OHLC technique also known

as price candlesticks [36]. Given a timeframe length (e.g. 1 h), the trade data are binned into their respective timeframes based on their execution time and then for each bin the first (open), highest, lowest and last (close) price is calculated. This provides with a useful illustration of the price of an asset across time.

The dataset we contains 66 FOREX pairs including widely used currencies such as Euro, US Dollar, and Pound Sterling and is available at [37].

4. Stationary feature and label extraction

The raw LOB data cannot be directly used for any ML task without some kind of preprocessing. The order volume values can be gathered for all stocks' LOBs and normalized together, since they are expected to follow the same distribution. However, this is not true for price values, since the value of a stock or asset may fluctuate and increase with time to never before seen levels. This means that the statistics of the price values can change significantly with time, rendering the price time series non-stationary.

Simply normalizing all the price values will not resolve the non-stationarity, since there will always be unseen data that may change the distribution of values to ranges that are not present in the current data. We present two solutions for this problem, one used in past work where normalization is applied constantly using past available statistics and a new approach to completely convert the price data to stationary values.

4.1. Input normalization

The most common normalization scheme is standardization (z-score):

$$x_{\text{norm}} = \frac{x - \bar{x}}{\sigma_{\bar{x}}}, \tag{1}$$

where x is a feature to be normalized, $\bar{x}$ is the mean and $\sigma_{\bar{x}}$ is the standard deviation across all samples. Such normalization is separately applied to the order size values and the price values. Using this kind of "global" normalization allows the preservation of the different scales between prices of different stocks, which we are trying to avoid. The solution presented in [30,31] is to use z-score to normalize each stock-day worth of data with the means and standard deviations calculated using previous day's data of the same stock. This way a major problem is avoided which is the distribution shift in stock prices, that can be caused by events such as stock splits or the large shifts in price that can happen over longer periods of time.

Unfortunately this presents another important issue for learning. The difference between the price values in different LOB levels are almost always minuscule. Since all the price levels are normalized using z-score with the same statistics, extracting features at that scale is hard. In this work we propose a novel approach to remedy this problem. Instead of normalizing the raw values of the LOB depth, we modify the price values to be their percentage difference to the current mid price of the Order Book. This removes the non-stationarity from the price values, makes the feature extraction process easier and significantly improves the performance of ML models, as it is also experimentally demonstrated in Section 6. To compensate for the removal of the price value itself we add an extra value to each LOB depth sample which is the percentage change of the mid price since the previous event.

The mid-price is defined as the mid-point between the best bid and the best ask prices at time t by

$$p_m^{(1)}(t) = \frac{p_a^{(1)}(t) + p_b^{(1)}(t)}{2}. \tag{2}$$

Table 1
Brief description of each proposed stationary feature.

Feature	Description
Price level difference	The difference of each price level to the current mid price, see Eqs. (3), (4) $p'^{(i)}(t) = \frac{p^{(i)}(t)}{p_m(t)} - 1$
Mid price change	The change of the current mid price to the mid price of the previous time-step, see Eq. (5) $p'_m(t) = \frac{p_m(t)}{p_m(t-1)} - 1$
Depth size cumsum	Total depth at each price level, see Eqs. (6), (7) $\nu'^{(k)}(t) = \sum_{i=1}^{k} \nu^{(i)}(t)$

where $p_a^{(1)}(t)$ and $p_b^{(1)}(t)$ are the ask and bid price present in the first level of the orderbook at time t. Consequently the mid point $p_m^{(1)}(t)$ is calculated by averaging the aforementioned price levels.

Let

$$p'^{(i)}_a(t) = \frac{p_a^{(i)}(t)}{p_m(t)} - 1, \tag{3}$$

$$p'^{(i)}_b(t) = \frac{p_b^{(i)}(t)}{p_m(t)} - 1, \tag{4}$$

and

$$p'_m(t) = \frac{p_m(t)}{p_m(t-1)} - 1. \tag{5}$$

Eqs. (3) and (4) serve as statistic features that represent the proportional difference between ith price and the mid-price at time t. Eq. (2), on the other hand, serves as a dynamic feature that captures the proportional mid-price movement over the time period (that is, it represents asset's return in terms of mid-prices).

We also use the cumulative sum of the sizes of the price levels as a feature, also know as Total Depth:

$$\nu'^{(k)}_a(t) = \sum_{i=1}^{k} \nu_a^{(i)}(t) \tag{6}$$

$$\nu'^{(k)}_b(t) = \sum_{i=1}^{k} \nu_b^{(i)}(t) \tag{7}$$

where $\nu_a^{(i)}(t)$ is number of outstanding limit order at the ith best ask price level and $\nu_b^{(i)}(t)$ is number of outstanding limit order at the bth best ask price level.

The proposed stationary features are briefly summarized in Table 1. After constructing these three types of stationary features, each of them is separately normalized using standardization (z-score), as described in (1), and concatenated into a single feature vector $\mathbf{x}_t$, where t denotes the time-step.

The input used for the time-aware models, such as the CNN, LSTM and CNN-LSTM, is the sequence of vectors $\mathbf{X} = \{\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_w\}$, where w is the number of total number of events each one represented by a different time-step input. For the models that need all the input into a single vector, such as the SVM and MLP models, the matrix $\mathbf{X}$ is flatten into a single dimension so it can be used as input for these models.

Similarly for the OHLC FOREX data that is used, the features that are utilized consist of the percentage change of each of the OHLC price levels for each time-step, in the same manner the mid price change is calculated for the LOB dataset.

4.2. Labels

The proposed models aim to predict the future movements of the mid price. Therefore, the ground truth labels must be appropriately generated to reflect the future mid price movements. Note that the mid price is a "virtual" value and no order can be guaranteed to immediately executed if placed at that exact price. However being able to predict its upwards or downwards movement provides a good estimate of the price of the future orders. A set of discrete choices must be constructed from our data to use as target for our classification models. The labels for describing the movement denoted by $y_t \in \{-1, 0, 1\}$, where t denotes the time-step.

Simply using $p_m(t+k) > p_m(t)$ to determine the upward direction of the mid price would introduce unmanageable amount of noise, since the smallest change would be registered as an upward or downward movement. To remedy this, in our previous work [30,31] the noisy changes of the mid price were filtered by employing two averaging filters. One averaging filter was used on a window of size k of the past values of the mid price and another averaging was applied on a future window k:

$$m_b(t) = \frac{1}{k+1} \sum_{i=0}^{k} p_m(t-i) \tag{8}$$

$$m_a(t) = \frac{1}{k} \sum_{i=1}^{k} p_m(t+i) \tag{9}$$

where p_t is the mid price as described in Eq. (2). The label l_t, that expresses the direction of price movement at time t, is extracted by comparing the previously defined quantities (m_b and m_a). However, using the m_b values to create labels for the samples, as in [30,31], is making the problem significantly easier and predictable due to the slower adaptation of the mean filter values to sudden changes in price. Therefore, in this work we remedy this issue by replacing m_b with the mid price. Therefore, the labels are redefined as:

$$l_t = \begin{cases} 1, & \text{if } \dfrac{m_a(t)}{p_m(t)} > 1+\alpha \\ -1, & \text{if } \dfrac{m_a(t)}{p_m(t)} < 1-\alpha \\ 0, & \text{otherwise} \end{cases} \tag{10}$$

where α is the threshold that determines how significant a mid price change $m_a(t)$ must be in order to label the movement as upward or downward. Values that do not satisfy this inequality are considered as insignificant and are labelled as having no price movement, or in other words being "stationary". The resulting labels present the trend to be predicted. This process is applied across all time-steps of the dataset to produce labels for all the depth samples.

The FOREX price dataset labels are created in the same way as Eq. (10) by overloading the notation of $p_m(t)$ to be the close price at time-step t. The resulting calculation of the direction for the FOREX price is defined as the ratio of the current close price to the average of the next k close prices.

5. Machine learning models

In this section we explain the particular inner workings of the CNN and LSTM models that are used and present how they are combined to form the proposed CNN-LSTM model. The technical details of each model are explained along with the employed optimization procedure.

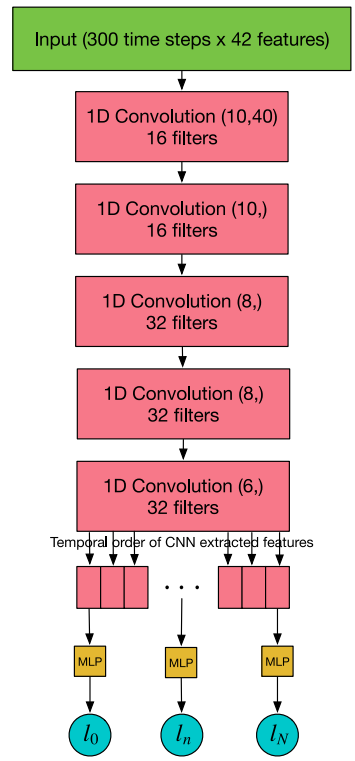

Fig. 1. A visual representation of the evaluated CNN model. Each layer includes the filter input size and the number of filters used.

5.1. Convolutional Neural Networks

Convolutional Neural Networks (CNNs) consist of the sequential application of convolutional and pooling layers usually followed by some fully connected layers, as shown in Fig. 1. Each convolutional layer i is equipped with a set of filters $\mathbf{W}_i \in \mathbb{R}^{S \times D \times N}$ that is convolved with an input tensor, where S is the number of used filters, D is the filter size, and N is the number of the input channels. The input tensor $\mathbf{X} \in \mathbb{R}^{(B \times T \times F)}$ is consisted by the temporally ordered features described in Section 4.1, where B is the batch size, T is the number of time-steps and F is the number of features per time-step.

In this work we leverage the causal padding introduced in [3] to avoid using future information to produce features for the current time-step. Using a series of convolutional layers allows for capturing the fine temporal dynamics of the time series as well as correlating temporally distant features. After the last convolutional/pooling layer a set of fully connected layers are used to classify the input time series. The network's output expresses the categorical distribution for the three direction labels (upward, downward and stationary), as described in (10), for each time-step.

We also employ a temporal batching technique, similar to the one used in LSTMs, to increase the computational efficiency and reduce memory requirements of our experiments when training with CNNs. Given the above described input tensor $\mathbf{X}$ and convolution filters $\mathbf{W}_i$ the last convolution produces a tensor with dimensions (B, T, S, N), which in most uses cases is flattened to a tensor of size $(B, T \times S \times N)$ before being fed to a fully connected layer. Instead we retain the temporal ordering by only reducing the tensor to dimension $(B, T, S \times N)$. An identical fully connected network with a softmax output is applied for each $S \times N$ vectors leading to T different predictions.

Since we are using causal convolutions with "full" padding, all the convolutional layers produce the same time-steps T, hence

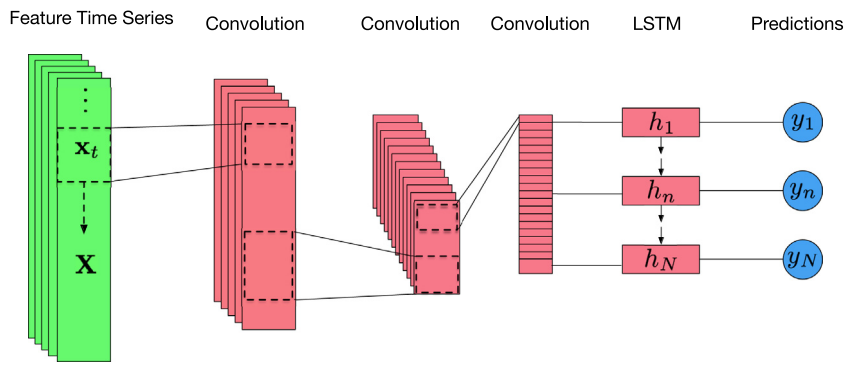

Fig. 2. CNN-LSTM model.

we do not need to worry about label alignment to the correct time-step. Also the causal convolutions ensure that no information from the future leaks to past time-step filters. This technique reduces the receptive field of the employed CNN, but this can be easily remedied by using a greater number of convolutional layers and/or a larger filter size D.

5.2. Long short term memory recurrent neural networks

One of the most appropriate Neural Network architectures to apply on time series is the Recurrent Neural Network (RNN) architecture. Although powerful in theory, this type of network suffers from the vanishing gradient problem, which makes the gradient propagation through a large number of steps impossible. An architecture that was introduced to solve this problem is the Long Short Term Memory (LSTM) networks [23]. This architecture protects its hidden activation from the decay of unrelated inputs and gradients by using gated functions between its "transaction" points. The protected hidden activation is the "cell state" which is regulated by said gates in the following manner:

$$\mathbf{f}_t = \sigma(\mathbf{W}_{xf} \cdot \mathbf{x} + \mathbf{W}_{hf} \cdot \mathbf{h}_{t-1} + \mathbf{b}_f) \tag{11}$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_{xi} \cdot \mathbf{x} + \mathbf{W}_{hi} \cdot \mathbf{h}_{t-1} + \mathbf{b}_i) \tag{12}$$

$$\mathbf{c}'_t = tanh(\mathbf{W}_{hc} \cdot \mathbf{h}_{t-1} + \mathbf{W}_{xc} \cdot \mathbf{x}_t + \mathbf{b}_c) \tag{13}$$

$$\mathbf{c}_t = \mathbf{f}_t \cdot \mathbf{c}_{t-1} + \mathbf{i}_t \cdot \mathbf{c}'_t \tag{14}$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_{oc} \cdot \mathbf{c}_t + \mathbf{W}_{oh} \cdot \mathbf{h}_{t-1} + \mathbf{b}_o) \tag{15}$$

$$\mathbf{h}_t = \mathbf{o}_t \cdot \sigma(\mathbf{c}_t) \tag{16}$$

where $\mathbf{f}_t$, $\mathbf{i}_t$ and $\mathbf{o}_t$ are the activations of the input, forget and output gates at time-step t, which control how much of the input and the previous state will be considered and how much of the cell state will be included in the hidden activation of the network. The protected cell activation at time-step t is denoted by $\mathbf{c}_t$, whereas $\mathbf{h}_t$ is the activation that will be given to other components of the model. The matrices $\mathbf{W}_{xf}, \mathbf{W}_{hf}, \mathbf{W}_{xi}, \mathbf{W}_{hi}, \mathbf{W}_{hc}, \mathbf{W}_{xc}, \mathbf{W}_{oc}, \mathbf{W}_{oh}$ are used to denote the weights connecting each of the activations with the current time-step inputs and the previous time-step activations.

5.3. Combination of models (CNN-LSTM)

We also introduce a powerful combination of the two previously described models. The CNN model is identically applied as described in Section 5.1, using causal convolutions and temporal batching to produce a set of features for each time-step. In essence the CNN acts as the feature extractor of the LOB depth time series, which produces a new time series of features with the same length as the original one, with each of them having time-steps corresponding to one another.

An LSTM layer is then applied on the time series produced by the CNN, and in turn produces a label for each time-step. This works in a very similar way to the fully connected layer described in 5.1 for temporal batching, but instead of the Fully Connected layer the LSTM allows the model to incorporate the features from past steps. The model architecture is visualized in Fig. 2.

5.4. Optimization

The parameters of the models are learned by minimizing the categorical cross entropy loss defined as:

$$\mathcal{L}(\mathbf{W}) = -\sum_{i=1}^{L} y_i \cdot \log \hat{y}_i, \tag{17}$$

where L is the number of different labels and the notation $\mathbf{W}$ is used to refer to the parameters of the models. The ground truth vector is denoted by $\mathbf{y}$, while $\hat{\mathbf{y}}$ is the predicted label distribution. The loss is summed over all samples in each batch. Due to the unavoidable class imbalance of this type of dataset, a weighted loss is employed to improve the mean recall and precision across all classes:

$$\mathcal{L}(\mathbf{W}) = -\sum_{i=1}^{L} c_{y_i} \cdot y_i \cdot \log \hat{y}_i, \tag{18}$$

where c_{y_i} is the assigned weight for the class of y_i. The individual weight c_i assigned to each class i is calculated as:

$$c_i = \frac{|\mathcal{D}|}{n \cdot |\mathcal{D}_i|}, \tag{19}$$

where $|\mathcal{D}|$ is the total number of samples in our dataset $\mathcal{D}$, n is the total number of classes (which in our case is 3) and $\mathcal{D}_i$ is set of samples from our dataset that have been labelled to belong in class i.

The most commonly used method to minimize the loss function defined in (18) and learn the parameters $\mathbf{W}$ of the model is gradient descent [38]:

$$\mathbf{W}' = \mathbf{W} - \eta \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{W}} \tag{20}$$

where $\mathbf{W}'$ are the parameters of the model after each gradient descent step and η is the learning rate. In this work we utilize the RMSProp optimizer [39], which is an adaptive learning rate method and has been shown to improve the training time and performance of DL models.

The LSTM, CNN and CNN-LSTM models along with all the training algorithms were developed using Keras [40], which is a framework built on top of the Tensorflow library [41].

Table 2
Example of sample distribution across classes depending on α for prediction horizon $k = 100$.

α	Down	Stationary	Up
1.0×10^{-5}	0.39	0.17	0.45
2.0×10^{-5}	0.38	0.19	0.43
5.0×10^{-5}	0.35	0.25	0.41
1.0×10^{-4}	0.30	0.33	0.36
2.0×10^{-4}	0.23	0.49	0.28
3.0×10^{-4}	0.18	0.60	0.22
3.5×10^{-4}	0.15	0.66	0.19

6. Experimental evaluation

In this section we present the experimental evaluation of our proposed CNN-LSTM model compared with simpler models such as SVM, MLP, CNN and LSTM. We also present the performance improvements of the proposed LOB stationary features compared to raw LOB data. Finally we examine the performance of the proposed model when applied to the subsampled OHLC price data.

6.1. Limit Order Book

All the models were tested for step sizes $k = 10, 50, 100$, and 200 in Eq. (9), where the α value for each was set at 2×10^{-5}, 9×10^{-5}, 3×10^{-4} and 3.5×10^{-4} respectively. The parameter α was chosen in conjunction with the future horizon with the aim to have relatively balanced distribution of labels across classes. In a real trading scenario it is not possible to have a profitable strategy that creates as many trade signals as "no-trade" signals, because it would accumulate enormous commission costs. For that reason α is selected with the aim to get a logical ratio of about 20% long, 20% short and 60% stationary labels. The effect of varying the parameter α on the class distribution of labels is shown in Table 2. Note that increasing the α allows for reducing the number of trade signals which should be changed depending on the actual commission and slippage costs that are expected to occur.

We tested the CNN and LSTM models using the raw features and the proposed stationary features separately and compared the results. Both the raw features and the stationary ones where normalized using z-score normalization. The architecture of the three models that were tested is described below.

The proposed CNN model consists of the following sequential layers:

1. 1D Convolution with 16 filters of size (10, 42)
2. 1D Convolution with 16 filters of size (10,)
3. 1D Convolution with 32 filters of size (8,)
4. 1D Convolution with 32 filters of size (6,)
5. 1D Convolution with 32 filters of size (4,)
6. Fully connected layer with 32 neurons
7. Fully connected layer with 3 neurons

The activation function used for all the convolutional and fully connected layer of the CNN is the Parametric Rectifying Linear Unit (PRELU) [42]. The last layer uses the softmax function for the prediction of the probability distribution between the different classes. All the convolutional layers are followed by a Batch Normalization (BN) layer after them.

The LSTM network uses 32 hidden neurons followed by a feed-forward layer with 64 neurons using Dropout and PRELU as activation function. Experimentally we found out that the hidden layer of the LSTM should contain 64 or less hidden neurons to avoid over-fitting the model. Experimenting with higher number of hidden neurons would be feasible if the dataset was even larger.

Table 3
Experimental results for different prediction horizons k. The values that are reported are the mean of each metric for the last 20 training epochs.

Feature type	Model	Mean recall	Mean precision	Mean F1	Cohen's κ
Prediction horizon $k = 10$					
Raw values	SVM	0.35	0.43	0.33	0.04
	MLP	0.34	0.34	0.09	0.00
	CNN	0.51	0.42	0.38	0.14
	LSTM	0.49	0.41	0.35	0.12
Stationary features	SVM	0.33	**0.46**	0.30	0.011
	MLP	0.34	0.35	0.09	0.00
	CNN	0.53	0.46	0.46	0.21
	LSTM	**0.55**	0.46	0.43	0.20
	CNN-LSTM	**0.55**	**0.46**	**0.47**	**0.23**
Prediction horizon $k = 50$					
Raw Values	SVM	0.35	0.41	0.32	0.03
	MLP	0.41	0.38	0.21	0.04
	CNN	0.50	0.42	0.37	0.13
	LSTM	0.46	0.40	0.34	0.10
Stationary features	SVM	0.39	0.41	0.38	0.09
	MLP	0.49	0.43	0.38	0.14
	CNN	0.53	0.45	0.45	0.21
	LSTM	**0.56**	0.46	0.45	0.22
	CNN-LSTM	**0.56**	**0.47**	**0.47**	**0.24**
Prediction horizon $k = 100$					
Raw values	SVM	0.35	0.46	0.33	0.05
	MLP	0.45	0.39	0.26	0.06
	CNN	0.49	0.42	0.37	0.12
	LSTM	0.45	0.39	0.34	0.09
Stationary features	SVM	0.36	0.46	0.35	0.07
	MLP	0.50	0.43	0.39	0.14
	CNN	0.51	0.46	0.44	0.20
	LSTM	**0.56**	0.46	0.46	0.22
	CNN-LSTM	0.54	**0.47**	**0.47**	**0.23**
Prediction horizon $k = 200$					
Raw values	SVM	0.35	0.44	0.31	0.04
	MLP	0.44	0.40	0.32	0.08
	CNN	0.47	0.43	0.39	0.14
	LSTM	0.42	0.39	0.36	0.08
Stationary features	SVM	0.38	0.46	0.36	0.10
	MLP	0.49	0.45	0.42	0.17
	CNN	**0.54**	0.45	0.44	0.20
	LSTM	0.53	0.47	0.47	0.23
	CNN-LSTM	0.51	**0.48**	**0.49**	**0.25**

Finally the CNN-LSTM model applies the convolutional feature extraction layers on the input and then feeds them in the correct temporal order to an LSTM model. The CNN component is comprised of the following layers:

1. 1D Convolution with 16 filters of size (5, 42)
2. 1D Convolution with 16 filters of size (5,)
3. 1D Convolution with 32 filters of size (5,)
4. 1D Convolution with 32 filters of size (5,)

Note that the receptive field of each convolutional filter in the CNN module is smaller that the standalone CNN, since the LSTM can capture most of the information from past time-steps. The LSTM module has the same exact architecture as the standalone LSTM. A visual representation of this CNN-LSTM model is shown in Fig. 2. Likewise, PRELU is the activation function used for the CNN and the fully connected layers, while the softmax function is used for the output layer of the network to predict the probability distribution of the classes.

One recurring effect we observe when training LSTM networks on LOB data is that for the first steps of observation the

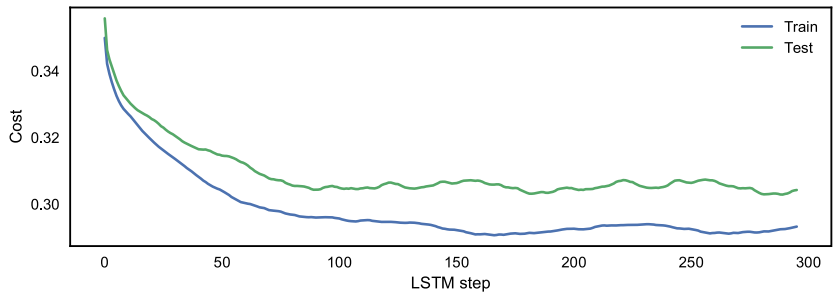

Fig. 3. Mean cost per recurrent step of the LSTM network.

predictions y_i yield a bigger cross entropy cost, meaning worse performance in our metrics. We run a set of experiments where the LSTM was trained for all the steps of the input windows T. The resulting mean cost per time-step can be observed in Fig. 3. As a result, trying to predict the price movement using insufficient past information is not possible and should be avoided since it leads to noisy gradients. To avoid this, a "burn-in" input is initially used to build its initial perception of the market before actually making correct decisions. In essence the first "burn-in" steps of the input are skipped, by not allowing any gradient to alter our model until after the 100th time-step. We also apply the same method to the CNN-LSTM model.

For training the models, the dataset is split as follows. The first 7 days of each stock are used to train the models, while the final 3 days are used as test data. The experiments were conducted for 4 different prediction horizons k, as defined in (9) and (10).

Performance is measured using Cohen's kappa [43], which is used to evaluate the concordance between sets of given answers, taking into consideration the possibility of random agreements happening. The mean recall, mean precision and mean F1 score between all 3 classes is also reported. Recall is the number of true positives samples divided by the sum of true positives and false negatives, while precision is the number of true positive divided by the sum of true positives and false positives. F1 score is the harmonic mean of the precision and recall metrics.

The results of the experiments are shown in Table 3. The results are compared for the models trained on the raw price features with the ones trained using the extracted stationary features. The proposed stationary features across all the presented models achieve greater performance than using the raw (normalized) values, leading to an average relative increase of the Cohen's κ metric by a factor of 1.55 for CNNs 2.29 for LSTMs. For the LSTM model the stationary features achieve an even more impressive improvement of the Cohen's κ metric of up to a factor of 2.8 in some cases (k = 200).

We also trained a Linear SVM model and a simple MLP model and compared them to the DL models. The SVM model was trained using Stochastic Gradient Descent since the size of the dataset is too large to use a regular Quadratic Programming solver. The SVM model implementation is provided by the sklearn library [44]. The MLP model consists of three fully connected layers with sizes 128, 64, 32, and PRELU as activations for each layers. Dropout is also used to avoid overfitting and the softmax activation function was used in the last layer.

Since both the SVM and the MLP models cannot iterate over time-steps to gain the same amount of information as the CNN and LSTM-based models, a window of 50 depth events is used and is flattened into a single sample. This process is applied in a rolling fashion for all the dataset to generate a dataset upon which the two models can be trained. One important note is the training fluctuations that are observed in Fig. 4, which are caused by the great class imbalance. Similar issues where observed in initial experiments with CNN and LSTM models but using the weighted loss described in 5.4 the fluctuations subsided.

Table 4
CNN sensitivity analysis for prediction horizons $k = 200$.

Filter sizes	Mean recall	Mean precision	Mean F1	Cohen's κ
CNN				
8–8–8–16–16	0.515	0.465	0.446	0.207
16–16–16–32–32	0.516	0.466	0.457	0.216
24–24–24–64–64	0.511	0.464	0.459	0.211
CNN-LSTM				
8–8–16–16	0.507	0.469	0.473	0.241
16–16–32–32	0.510	0.469	0.474	0.238
24–24–64–64	0.513	0.465	0.468	0.235

The proposed stationary price features significantly outperform the raw price features for all the tested models. This can be attributed to a great extent to the stationary nature of the proposed features. The employed price differences provide an intrinsically stationary and normalized price measure that can be directly used. This is in contrast with the raw price values that requires careful normalization to ensure that their values remain into a reasonable range and suffer for significantly non-stationarity issues when the price increases to levels not seen before. By converting the actual prices to the their difference to the mid price and normalize that, this important feature is exaggerated to avoid being suppressed by the much larger price movements through time. The proposed combination model CNN-LSTM also outperforms its separated individual component models as shown in Fig. 4 and Table 3 showing that it can better handle the LOB data and use them to take advantage of the microstructure existing within the data to produce more accurate predictions.

6.1.1. Sensitivity analysis

To ensure the resulting performance of the models containing CNN are not greatly affected by altering the model parameters, we conduct a sensitivity analysis for the number of filters used on every CNN layer. We vary the number of convolutional filters by multiples of 2 and examine the resulting performances. The resulting performance are presented in Table 4.

The performance across the different filters stays within reasonable distance, confirming that the resulting performance is not affected by minor changes to the model parameters.

6.1.2. Normalization selection

To ensure that the most appropriate normalization method is selected, a set of experiments was conducted to evaluate the effect of using different normalization approaches. The results of these experiments are reported in Table 5. The best performing normalization is the z-score standardization, so it was selected to normalize the inputs for the rest of the conducted experiments.

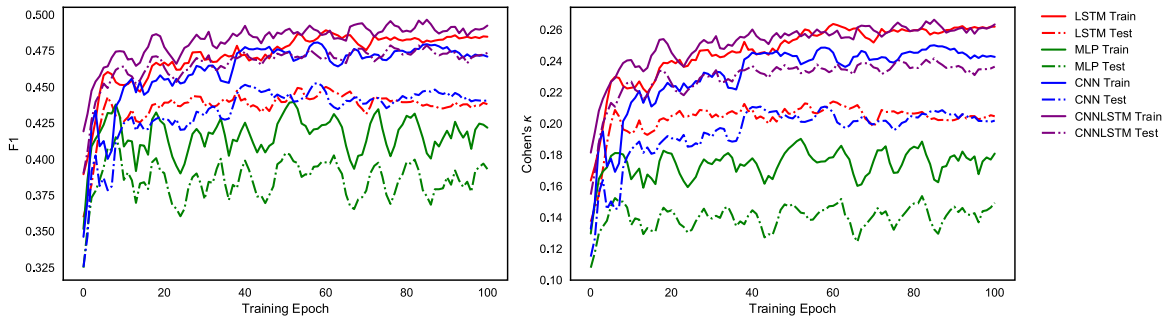

Fig. 4. F1 and Cohen's κ metrics during training for prediction horizon $k = 100$. Plots are smoothed with a mean filter with window = 3 to reduce fluctuations.

Table 5
Comparison between different normalization methods. A CNN with 16–16–16–32–32 convolutional filters was employed. The prediction horizon was set to $k = 200$. Scale normalization scales the feature values to the specified range.

Normalization	Mean Recall	Mean Precision	Mean F1	Cohen's k
scale [0,1]	0.404	0.401	0.267	0.073
scale [−1,1]	0.433	0.432	0.316	0.103
z-score	0.516	0.466	0.454	0.216

Table 6
Forex data resulting performance. The prediction horizon k is used to calculate the $m_a(t)$ average filter based on the future k OHLC price samples.

Model	Mean recall	Mean precision	Mean F1	Cohen's κ
Prediction horizon $k = 5$				
CNN	0.441	0.444	0.417	0.182
LSTM	0.446	0.448	0.438	0.191
CNN-LSTM	**0.455**	**0.454**	**0.454**	**0.202**
Prediction horizon $k = 10$				
CNN	0.448	0.449	0.447	0.199
LSTM	0.448	0.449	0.445	0.202
CNN-LSTM	**0.452**	**0.452**	**0.451**	**0.207**
Prediction horizon $k = 15$				
CNN	0.444	0.443	0.443	0.198
LSTM	0.441	0.443	0.439	0.199
CNN-LSTM	**0.447**	**0.446**	**0.446**	**0.203**
Prediction horizon $k = 20$				
CNN	0.437	0.435	0.435	0.193
LSTM	**0.444**	**0.445**	**0.444**	**0.204**
CNN-LSTM	0.441	0.441	0.439	0.201

6.2. OHLC FOREX data

The same model architecture that was used for the LOB data was also applied to the FOREX price data to compare the respective models effectiveness on a dataset with lower density of data for the units of time. In the FOREX dataset case the OHLC price points are sampled with a window size of 1 h and the features consist of the percentage returns of the OHLC prices between time-steps.

The resulting performances are presented in Table 6. It is pretty clear that the LSTM performs very similarly to our proposed CNN-LSTM model, although our models manages to outperform it slightly, while in the previous LOB dataset the CNN-LSTM consistently outperformed the models with larger margins in important metrics such as Cohen's κ. This relative performance discrepancy between the datasets can be attributed to the greater potential for feature extraction that exists in the high resolution LOB data versus the subsampled OHLC price data where a lot of information has been stripped.

7. Conclusion

In this paper we proposed a novel method for extracting stationary features from raw LOB data, suitable for use with different DL models. Using different ML models, i.e., SVMs, MLPs, CNNs and LSTMs, it was experimentally demonstrated that the proposed features significantly outperform the raw price features. The proposed stationary features achieve this by making the difference between the prices in the LOB depth the main metric instead of the price itself, which usually fluctuates much more through time than the price level within the LOB. A novel combined CNN-LSTM model was also proposed for time series predictions and it was demonstrated that exhibits more stable behaviour and leads to better results that the CNN and LSTM models.

There are several interesting future research directions. As with all the DL application, more data would enable the use of bigger models that would not be at risk of being overtrained as it was observed in this work. An RNN-type of network could be also used to perform a form of "intelligent" r-sampling extracting useful features from a specific and limited time-interval of depth events, which would avoid losing information and allow for the later models produce prediction for a certain time period and not for a number of following events. Another important addition would be an attention mechanism [19,45], which would allow for the better observation of the features by the network allowing it to ignore noisy parts of the data and use only the relevant information.

Ethical approval

This article does not contain any studies with human participants or animals performed by any of the authors.

CRediT authorship contribution statement

Avraam Tsantekidis: Software, Writing - original draft. **Nikolaos Passalis:** Software, Writing - original draft. **Anastasios Tefas:** Supervision, Writing - review & editing. **Juho Kanniainen:** Supervision, Writing - review & editing. **Moncef Gabbouj:** Supervision, Writing - review & editing. **Alexandros Iosifidis:** Supervision, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

For this work Avraam Tsantekidis was solely funded by a scholarship from the State Scholarship Foundation (IKY), Greece according to the "Strengthening Human Research Resources through the Pursuit of Doctoral Research" act, with resources from the "Human Resources Development, Education and Lifelong Learning 2014-2020" program. The research leading to these results has also received funding from the H2020 Marie Sklodowska-Curie Innovative Training Network, Finland BigData-Finance MSCA-ITN-ETN 675044 (http://bigdatafinance.eu), Training for Big Data in Financial Research and Risk Management.

References

[1] H. Deng, L. Zhang, X. Shu, Feature memory-based deep recurrent neural network for language modeling, Appl. Soft Comput. 68 (2018) 432–446.
[2] M. Larsson, Y. Zhang, F. Kahl, Robust abdominal organ segmentation using regional convolutional neural networks, Appl. Soft Comput. (2018).
[3] A. Van Den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, K. Kavukcuoglu, Wavenet: A generative model for raw audio, 2016, arXiv preprint arXiv:1609.03499.
[4] F. Black, M. Scholes, The pricing of options and corporate liabilities, J. Polit. Economy 81 (3) (1973) 637–654.
[5] R. Cont, S. Stoikov, R. Talreja, A stochastic model for order book dynamics, Oper. Res. 58 (3) (2010) 549–563.
[6] F.M. Bandi, R. Renò, Price and volatility co-jumps, J. Financ. Econ. 119 (1) (2016) 107–146.
[7] Y. Aït-Sahalia, P.A. Mykland, Estimating volatility in the presence of market microstructure noise: A review of the theory and practical considerations, in: Handbook of Financial Time Series, Springer, 2009, pp. 577–598.
[8] V. Vapnik, I. Guyon, T. Hastie, Support vector machines, Mach. Learn. 20 (3) (1995) 273–297.
[9] A. Kazem, E. Sharifi, F.K. Hussain, M. Saberi, O.K. Hussain, Support vector regression with chaos-based firefly algorithm for stock market price forecasting, Appl. Soft Comput. 13 (2) (2013) 947–958.
[10] T.-J. Hsieh, H.-F. Hsiao, W.-C. Yeh, Forecasting stock markets using wavelet transforms and recurrent neural networks: An integrated system based on artificial bee colony algorithm, Appl. Soft Comput. 11 (2) (2011) 2510–2525.
[11] L. Lei, Wavelet neural network prediction method of stock price trend based on rough set attribute reduction, Appl. Soft Comput. 62 (2018) 923–932.
[12] K. Michell, et al., A stock market risk forecasting model through integration of switching regime, ANFIS and GARCH techniques, Appl. Soft Comput. 67 (2018) 106–116.
[13] C.-F. Huang, A hybrid stock selection model using genetic algorithms and support vector regression, Appl. Soft Comput. 12 (2) (2012) 807–818.
[14] S. Galeshchuk, Neural networks performance in exchange rate prediction, Neurocomputing 172 (2016) 446–452.
[15] J. Heaton, N. Polson, J. Witte, Deep portfolio theory, 2016, arXiv preprint arXiv:1605.07230.
[16] L. Takeuchi, Y.-Y.A. Lee, Applying deep learning to enhance momentum trading strategies in stocks, 2013.
[17] R. Xiong, E.P. Nichols, Y. Shen, Deep learning stock volatility with google domestic trends, 2015, arXiv preprint arXiv:1512.04916.
[18] A. Graves, A.-r. Mohamed, G. Hinton, Speech recognition with deep recurrent neural networks, in: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing, 2013, pp. 6645–6649.
[19] K. Xu, J. Ba, R. Kiros, K. Cho, A.C. Courville, R. Salakhutdinov, R.S. Zemel, Y. Bengio, Show, attend and tell: Neural image caption generation with visual attention.
[20] J. Mao, W. Xu, Y. Yang, J. Wang, Z. Huang, A. Yuille, Deep captioning with multimodal recurrent neural networks (m-rnn), 2014, arXiv preprint arXiv:1412.6632.
[21] Y. Zhu, O. Groth, M. Bernstein, L. Fei-Fei, Visual7w: Grounded question answering in images, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 4995–5004.
[22] Y. LeCun, Y. Bengio, et al., Convolutional networks for images, speech, and time series, Handb. Brain Theory Neural Netw. 3361 (10) (1995) 1995.
[23] S. Hochreiter, J. Schmidhuber, Long short-term memory, Neural Comput. 9 (8) (1997) 1735–1780.
[24] D.-x. Niu, H.-f. Shi, D.D. Wu, Short-term load forecasting using bayesian neural networks learned by hybrid Monte Carlo algorithm, Appl. Soft Comput. 12 (6) (2012) 1822–1827.
[25] L. Xi, H. Muzhou, M.H. Lee, J. Li, D. Wei, H. Hai, Y. Wu, A new constructive neural network method for noise processing and its application on stock market prediction, Appl. Soft Comput. 15 (2014) 57–66.
[26] A.N. Kercheval, Y. Zhang, Modelling high-frequency limit order book dynamics with support vector machines, Quant. Finance 15 (8) (2015) 1315–1329.
[27] D.T. Tran, A. Iosifidis, J. Kanniainen, M. Gabbouj, Temporal attention-augmented bilinear network for financial time-series data analysis, IEEE Trans. Neural Netw. Learn. Syst. 30 (5) (2018) 1407–1418.
[28] A. Ntakaris, J. Kanniainen, M. Gabbouj, A. Iosifidis, Mid-price prediction based on machine learning methods with technical and quantitative indicators, 2019, arXiv preprint arXiv:1907.09452.
[29] D.T. Tran, M. Magris, J. Kanniainen, M. Gabbouj, A. Iosifidis, Tensor representation in high-frequency financial data for price change prediction, in: Computational Intelligence (SSCI), 2017 IEEE Symposium Series on, IEEE, 2017, pp. 1–7.
[30] A. Tsantekidis, N. Passalis, A. Tefas, J. Kanniainen, M. Gabbouj, A. Iosifidis, Forecasting stock prices from the limit order book using convolutional neural networks, in: Proceedings of the IEEE Conference on Business Informatics, Vol. 1, 2017, pp. 7–12.
[31] A. Tsantekidis, N. Passalis, A. Tefas, J. Kanniainen, M. Gabbouj, A. Iosifidis, Using deep learning to detect price change indications in financial markets, in: Proceedings of the European Signal Processing Conference, 2017, pp. 2511–2515.
[32] N. Passalis, A. Tsantekidis, A. Tefas, J. Kanniainen, M. Gabbouj, A. Iosifidis, Time-series classification using neural bag-of-features, in: Proceedings of the European Signal Processing Conference, 2017, pp. 301–305.
[33] D. Yang, Q. Zhang, Drift-independent volatility estimation based on high, low, open, and close prices, J. Bus. 73 (3) (2000) 477–492.
[34] A. Ntakaris, M. Magris, J. Kanniainen, M. Gabbouj, A. Iosifidis, Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods, J. Forecast. 37 (8) (2018) 852–866.
[35] M. Siikanen, J. Kanniainen, A. Luoma, What drives the sensitivity of limit order books to company announcement arrivals? Econom. Lett. 159 (2017) 65–68.
[36] S. Nison, Japanese Candlestick Charting Techniques: A Contemporary Guide to the Ancient Investment Techniques of the Far East, Penguin, 2001.
[37] Histdata, Financial historical data provider histdata.com, URL https://www.histdata.com.
[38] P.J. Werbos, Backpropagation through time: what it does and how to do it, Proc. IEEE 78 (10) (1990) 1550–1560.
[39] T. Tieleman, G. Hinton, Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude, COURSERA: Neural Netw. Mach. Learn. 4 (2) (2012) 26–31.
[40] F. Chollet, et al., Keras, GitHub, 2015, https://github.com/keras-team/keras.
[41] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, et al., TensorFlow: Large-scale machine learning on heterogeneous systems, 2015, Software available from tensorflow.org, URL https://www.tensorflow.org/.
[42] K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, in: Proceedings of the IEEE International Conference on Computer Vision, 2015, pp. 1026–1034.
[43] J. Cohen, A coefficient of agreement for nominal scales, Educ. Psychol. Meas. 20 (1) (1960) 37–46.
[44] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Scikit-learn: Machine learning in Python, J. Mach. Learn. Res. 12 (Oct) (2011) 2825–2830.
[45] K. Cho, A. Courville, Y. Bengio, Describing multimedia content using attention-based encoder-decoder networks, IEEE Trans. Multimed. 17 (11) (2015) 1875–1886.